

Имитационное моделирование

Отчёт по лабораторной работе № 6: модель SIR через сети Петри

Исаева Зарина Исмайлбековна
2026-09-07

Содержание

1. Информация о работе

Автор: Исаева Зарина Исмайлбековна
Студенческий билет: 1132232882
Дисциплина: Имитационное моделирование **Тема:** Реализация модели распространения эпидемии SIR с помощью сетей Петри.

2. Цель работы

Освоить представление SIR-модели как размеченной сети Петри, получить детерминированную и стохастическую траектории, исследовать чувствительность к коэффициентам переходов и подготовить воспроизводимые отчётные артефакты.

3. Задание

1. Создать модуль SIRPetri.jl на базе AlgebraicPetri.
2. Реализовать переходы заражения $S+I \rightarrow 2I$ и выздоровления $I \rightarrow R$ с интенсивностями βSI и γI .
3. Выполнить ODE-расчёт и прямой алгоритм Гиллеспи при $S_0=990$, $I_0=10$, $R_0=0$, $\beta=0.3$, $\gamma=0.1$, $T=100$.
4. Просканировать $\beta=0.1:0.05:0.8$, построить настоящую анимацию с шагом 0.2 и собрать сводку из ранее сохранённых CSV.
5. Создать параметрические варианты всех четырёх сценариев, литературные программы, выполненные ноутбуки и выходные документы.

4. Теоретическое введение

Классическая SIR-модель делит постоянную популяцию на восприимчивых S , инфицированных I и выбывших R [1]. В данной работе используется закон массового действия без нормирования на размер популяции, как указано в главе 6 практикума [2]:

$$\frac{dS}{dt} = -\beta SI, \frac{dI}{dt} = \beta SI - \gamma I, \frac{dR}{dt} = \gamma I.$$

Сеть Петри хранит структуру модели отдельно от численного метода. Места соответствуют S , I , R , а два перехода задают заражение и выздоровление. Структура построена объектом `LabelledPetriNet` пакета `AlgebraicPetri.jl` [3]. Детерминированная траектория решается `OrdinaryDiffEq.jl` [4], стохастическая — прямым алгоритмом Гиллеспи [5].

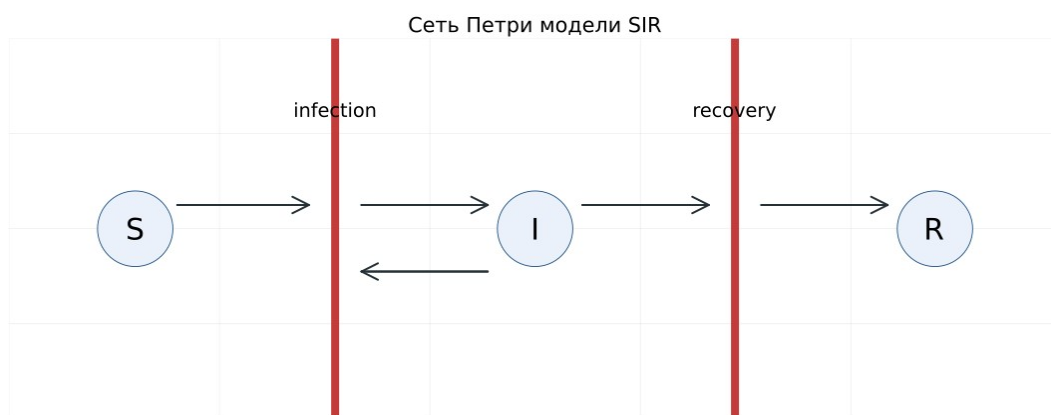


Рисунок 1: Структура сети Петри SIR

5. Организация проекта

Проект содержит исходные литературные сценарии, чистые Julia-файлы, выполненные ноутбуки, Quarto-документы, CSV-таблицы и графики. Все пути формируются относительно каталога проекта, поэтому репозиторий переносим.

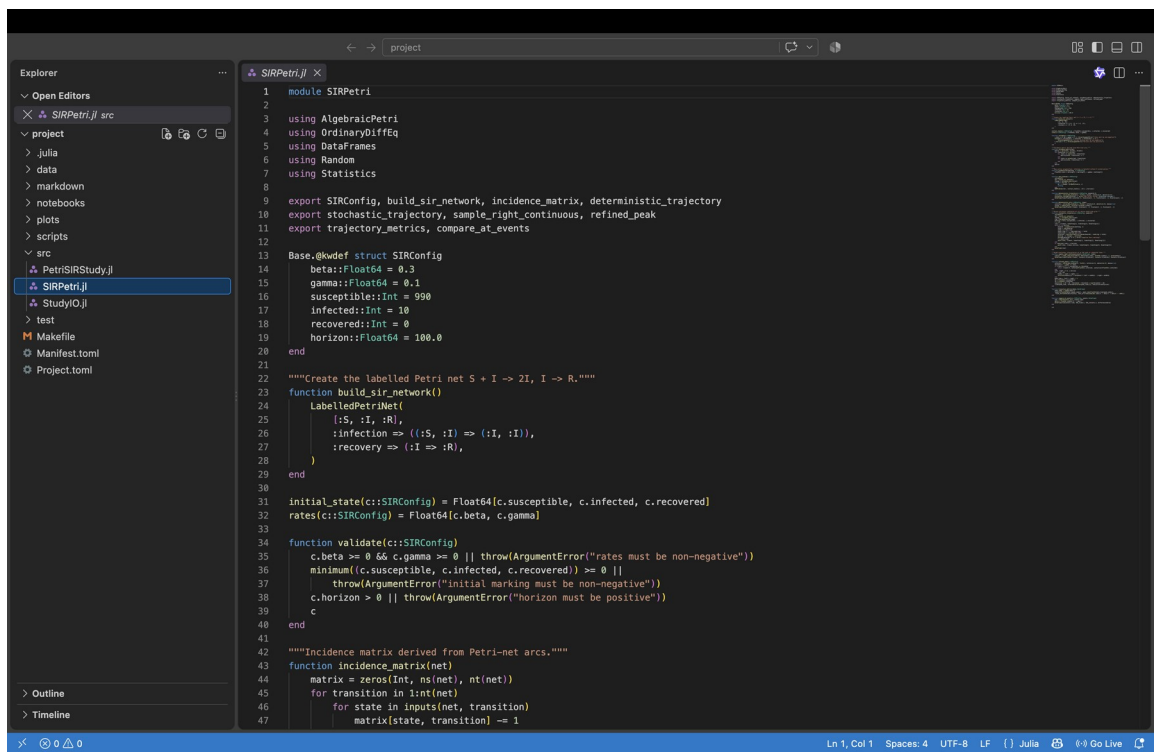


Рисунок 2: Структура результатов лабораторной работы

На рис. [Рисунок 3](#) показан исходный модуль в полноэкранном окне настоящего VS Code. В кадре отсутствуют учётные записи и пользовательские каталоги.

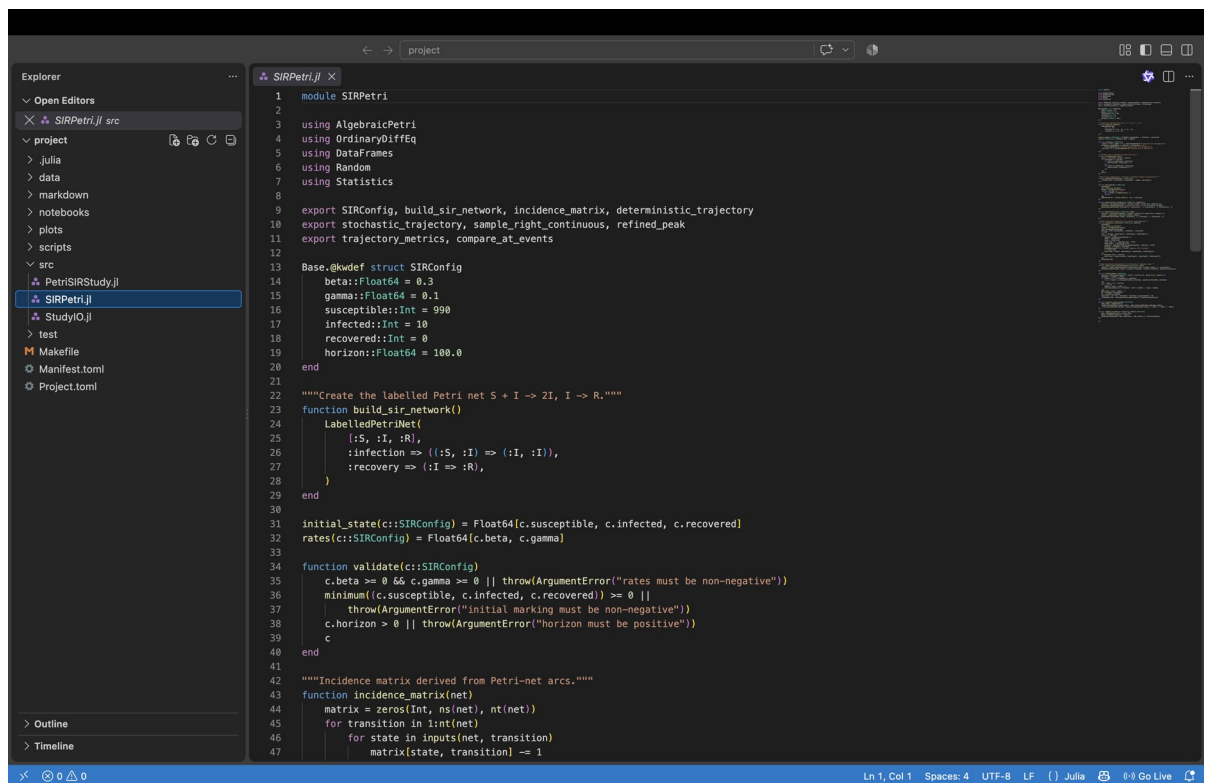


Рисунок 3: Модуль SIRPetri.jl в VS Code

Выполненные ноутбуки сохраняют исходный код, номера ячеек и рассчитанный вывод. На рис. [Рисунок 4](#) показан настоящий JupyterLab с базовым сценарием; ноутбук открыт только для чтения после завершения вычислений.

Рисунок 4: Выполненный `sirpetri_run.ipynb` в JupyterLab

6. Реализация модели

Матрица инцидентности вычисляется из дуг сети:

Место	infection	recovery
S	-1	0
I	1	-1
R	0	1

Так сохраняется инвариант $S+I+R=1000$. Ниже приведены полные листинги модулей, подготовительных программ и тестов.

6.1 Модуль сети Петри

module SIRPetri

```
using AlgebraicPetri
using OrdinaryDiffEq
using DataFrames
using Random
using Statistics
```

```
export SIRConfig, build_sir_network, incidence_matrix, deterministic_trajectory
export stochastic_trajectory, sample_right_continuous, refined_peak
export trajectory_metrics, compare_at_events
```

```
Base.@kwdef struct SIRConfig
    beta::Float64 = 0.3
    gamma::Float64 = 0.1
    susceptible::Int = 990
    infected::Int = 10
    recovered::Int = 0
```

```

    horizon::Float64 = 100.0
end

"""Create the labelled Petri net S + I -> 2I, I -> R."""
function build_sir_network()
    LabelledPetriNet(
        [:S, :I, :R],
        :infection => (:S, :I) => (:I, :I),
        :recovery => (:I => :R),
    )
end

initial_state(c::SIRConfig) = Float64[c.susceptible, c.infected, c.recovered]
rates(c::SIRConfig) = Float64[c.beta, c.gamma]

function validate(c::SIRConfig)
    c.beta >= 0 && c.gamma >= 0 || throw(ArgumentError("rates must be non-negative"))
    minimum((c.susceptible, c.infected, c.recovered)) >= 0 ||
        throw(ArgumentError("initial marking must be non-negative"))
    c.horizon > 0 || throw(ArgumentError("horizon must be positive"))
    c
end

"""Incidence matrix derived from Petri-net arcs."""
function incidence_matrix(net)
    matrix = zeros{Int, ns(net), nt(net)}
    for transition in 1:nt(net)
        for state in inputs(net, transition)
            matrix[state, transition] -= 1
        end
        for state in outputs(net, transition)
            matrix[state, transition] += 1
        end
    end
    matrix
end

"""Mass-action propensities; infection is beta*S*I without N normalization."""
function propensities(marking, c::SIRConfig)
    Float64[c.beta * marking[1] * marking[2], c.gamma * marking[2]]
end

function ode_problem(c::SIRConfig)
    validate(c)
    net = build_sir_network()
    change = incidence_matrix(net)
    function rhs!(du, u, _, _)
        du .= change * propensities(u, c)
        nothing
    end
    ODEProblem(rhs!, initial_state(c), (0.0, c.horizon))
end

function deterministic_trajectory(c::SIRConfig; saveat=0.5)
    solution = solve(ode_problem(c), Tsit5(); saveat, reltol=1e-9, abstol=1e-11)
    successful_retcodes(solution) || error("ODE solver failed: $(solution.retcodes)")
    DataFrame(time=Float64.(solution.t), S=solution[1, :], I=solution[2, :], R=solution[3, :])
end

function deterministic_at(c::SIRConfig, times)
    solution = solve(ode_problem(c), Tsit5(); reltol=1e-9, abstol=1e-11, dense=true)
    values = reduce(hcat, solution.(Float64.(times)))
    DataFrame(time=Float64.(times), S=values[1, :], I=values[2, :], R=values[3, :])
end

"""Direct Gillespie simulation on its native event-time grid."""
function stochastic_trajectory(c::SIRConfig; seed=123)
    validate(c)
    net = build_sir_network()

```

```

change = incidence_matrix(net)
rng = MersenneTwister(seed)
marking = Int[c.susceptible, c.infected, c.recovered]
t = 0.0
rows = [(time=t, S=marking[1], I=marking[2], R=marking[3])]
while t < c.horizon
  hazards = propensities(marking, c)
  total = sum(hazards)
  total <= 0 && break
  event_time = t - log(rand(rng)) / total
  event_time > c.horizon && break
  selected = searchsortedfirst(cumsum(hazards), rand(rng) * total)
  marking .+= change[:, selected]
  minimum(marking) >= 0 || error("negative Petri marking")
  t = event_time
  push!(rows, (time=t, S=marking[1], I=marking[2], R=marking[3]))
end
if rows[end].time < c.horizon
  push!(rows, (time=c.horizon, S=marking[1], I=marking[2], R=marking[3]))
end
DataFrame(rows)
end

"""Right-continuous interpolation of an SSA path at requested times."""
function sample_right_continuous(events::DataFrame, times)
  indices = clamp.(searchsortedlast.(Ref(events.time), Float64.(times)), 1, nrow(events))
  DataFrame(time=Float64.(times), S=events.S[indices], I=events.I[indices], R=events.R[indices])
end

function refined_peak(c::SIRConfig)
  solution = solve_ode_problem(c, Tsit5(); reltol=1e-11, abstol=1e-12, dense=true)
  threshold = c.gamma / c.beta
  if c.beta == 0 || c.susceptible <= threshold
    return (time=0.0, infected=Float64(c.infected), analytical=Float64(c.infected))
  end
  left, right = 0.0, c.horizon
  for _ in 1:80
    middle = (left + right) / 2
    solution(middle)[1] > threshold ? (left = middle) : (right = middle)
  end
  peak_time = (left + right) / 2
  s0 = Float64(c.susceptible)
  i0 = Float64(c.infected)
  analytical = i0 + s0 - threshold + threshold * log(threshold / s0)
  (time=peak_time, infected=solution(peak_time)[2], analytical=analytical)
end

function trajectory_metrics(data::DataFrame)
  peak_index = argmax(data.I)
  (peak_I=Float64(data.I[peak_index]), peak_time=Float64(data.time[peak_index]),
   final_R=Float64(data.R[end]), mass_error=maximum(abs.(data.S .+ data.I .- data.R .- 1000)))
end

function compare_at_events(c::SIRConfig, events::DataFrame)
  ode = deterministic_at(c, events.time)
  delta = Float64.(events.I) .- ode.I
  DataFrame(time=events.time, ODE_I=ode.I, SSA_I=events.I, difference=delta)
end

end

```

6.2 Работа с данными и графикой

module StudyIO

using CSV, DataFrames, Plots

export project_root, data_dir, plot_dir, ensure_outputs, save_csv, save_plot

export sir_plot, petri_diagram, animation_plot

```

project_root() = normpath(joinpath(@_DIR_, ".."))
data_dir(parts...) = joinpath(project_root(), "data", parts...)
plot_dir(parts...) = joinpath(project_root(), "plots", parts...)

```

```

function ensure_outputs(group)
    mkpath(data_dir(group))
    mkpath(plot_dir(group))
end

```

```

function save_csv(group, name, table)
    ensure_outputs(group)
    path = data_dir(group, name)
    CSV.write(path, table)
    path
end

```

```

function save_plot(group, name, figure)
    ensure_outputs(group)
    path = plot_dir(group, name)
    savefig(figure, path)
    path
end

```

```

function sir_plot(data; title="SIR dynamics", stepped=false)
    plot(data.time, data.S; label="S", color="#2468a2", linewidth=2.5,
        xlabel="Время", ylabel="Численность", title=title,
        seriestype=:stepped ? :steppost : :path, legend=:right)
    plot!(data.time, data.I; label="I", color="#cb3c33", linewidth=2.5,
        seriestype=:stepped ? :steppost : :path)
    plot!(data.time, data.R; label="R", color="#27864b", linewidth=2.5,
        seriestype=:stepped ? :steppost : :path)
end

```

```

function petri_diagram()
    plot(; xlim=(0, 10), ylim=(0, 4), axis=false, legend=false,
        size=(1100, 430), title="Сеть Петри модели SIR")
    scatter!([1.2, 5.0, 8.8], [2.0, 2.0, 2.0]; markersize=42,
        markercolor="#eaf1fb", markerstrokecolor="#174a7e")
    annotate!([(1.2, 2.0, text("S", 20, :black)),
        (5.0, 2.0, text("I", 20, :black)),
        (8.8, 2.0, text("R", 20, :black))])
    vline!([3.1, 6.9]; color="#c43b3b", linewidth=8, label=false)
    annotate!([(3.1, 3.25, text("infection", 13)), (6.9, 3.25, text("recovery", 13))])
    for (x1, x2, y) in [(1.6, 2.85, 2.25), (4.55, 3.35, 1.55),
        (3.35, 4.55, 2.25), (5.45, 6.65, 2.25), (7.15, 8.35, 2.25)]
        plot!([x1, x2], [y, y]; arrow=true, color="#263238", linewidth=2)
    end
    current()
end

```

```

function animation_plot(data, upto; title="SIR: динамика маркировки")
    segment = data[1:upto, :]
    left = sir_plot(segment; title=title)
    xlims!(left, 0, maximum(data.time)); ylims!(left, 0, 1000)
    right = bar(["S", "I", "R"], [data.S[upto], data.I[upto], data.R[upto]];
        color=["#2468a2", "#cb3c33", "#27864b"], legend=false,
        ylim=(0, 1000), title="t = $(round(data.time[upto], digits=1))")
    plot(left, right; layout=(1, 2), size=(1200, 520))
end

```

7. Выполнение обязательных экспериментов

7.1 Базовый эксперимент SIR через сеть Петри

Создаём сеть Петри, решаем детерминированную систему и выполняем один стохастический прогон методом Гиллеспи с фиксированным seed.

```
# # Базовый эксперимент SIR через сеть Петри
#
# Создаём сеть Петри, решаем детерминированную систему и выполняем один
# стохастический прогон методом Гиллеспи с фиксированным seed.

ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-run"
config = SIRConfig()
network = build_sir_network()
change = incidence_matrix(network)

deterministic = deterministic_trajectory(config; saveat=0.5)
stochastic = stochastic_trajectory(config; seed=123)
event_comparison = compare_at_events(config, stochastic)
peak = refined_peak(config)

save_csv(group, "sir_deterministic.csv", deterministic)
save_csv(group, "sir_stochastic_events.csv", stochastic)
save_csv(group, "event_time_comparison.csv", event_comparison)
save_csv(group, "refined_peak.csv", DataFrame(
    peak_time=[peak.time], numerical_peak=[peak.infected], analytical_peak=[peak.analytical],
    absolute_error=[abs(peak.infected - peak.analytical)])
save_csv(group, "incidence_matrix.csv", DataFrame(state=["S", "I", "R"],
    infection=change[:, 1], recovery=change[:, 2]))

save_plot(group, "petri-network.png", petri_diagram())
save_plot(group, "deterministic-dynamics.png",
    sir_plot(deterministic; title="Детерминированная модель:  $\beta=0.3$ ,  $\gamma=0.1$ "))
save_plot(group, "stochastic-dynamics.png",
    sir_plot(stochastic; title="Стохастическая модель: seed=123", stepped=true))

metrics = DataFrame(method=["ODE", "SSA"],
    peak_I=[peak.infected, maximum(stochastic.I)],
    peak_time=[peak.time, stochastic.time[argmax(stochastic.I)]],
    final_R=[deterministic.R[end], stochastic.R[end]],
    population_error=[maximum(abs.(deterministic.S + deterministic.I + deterministic.R - 1000)),
        maximum(abs.(stochastic.S + stochastic.I + stochastic.R - 1000))])
save_csv(group, "summary.csv", metrics)

println("Petri places/transitions: $(length(change[:, 1]))/$(length(change[1, :]))")
println("ODE rows: $(nrow(deterministic)); SSA events: $(nrow(stochastic))")
println("Refined peak: t=$(round(peak.time, digits=6)), I=$(round(peak.infected, digits=3))")
println("Analytical check error: $(abs(peak.infected - peak.analytical))")
```

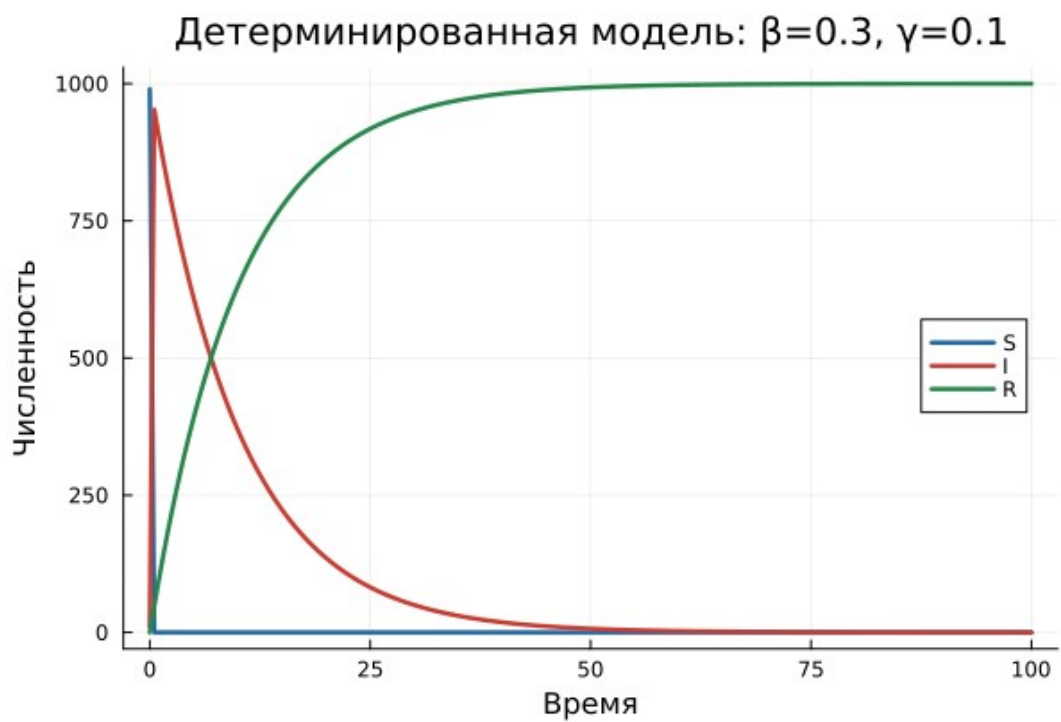



Рисунок 5: Детерминированная траектория

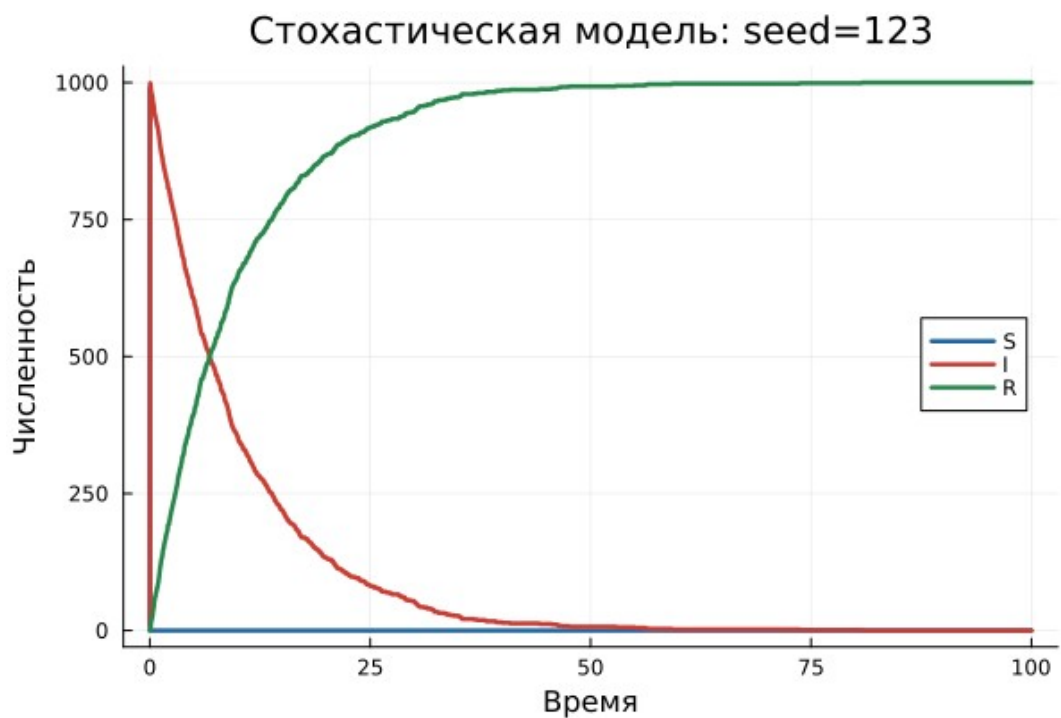


Рисунок 6: Стохастическая траектория

Максимум на сохранённой сетке $\Delta t=0.5$ не считается истинным ранним пиком. Поэтому время пика уточнено по условию $S=\gamma/\beta$, а значение проверено аналитическим инвариантом SIR.

7.2 Сканирование коэффициента заражения

Обязательная сетка $\beta=0.1:0.05:0.8$ при $\gamma=0.1$. Для каждого значения сохраняем максимум на сетке 0.5 и уточнённый ранний пик непрерывного решения.

```
# # Сканирование коэффициента заражения
#
# Обязательная сетка  $\beta=0.1:0.05:0.8$  при  $\gamma=0.1$ . Для каждого значения сохраняем
# максимум на сетке 0.5 и уточнённый ранний пик непрерывного решения.

ENV["GKSwstype"] = "100"
using DataFrames, Plots
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-scan"
rows = NamedTuple[]
for beta in 0.1:0.05:0.8
    config = SIRConfig(beta=Float64(beta), gamma=0.1)
    coarse = deterministic_trajectory(config; saveat=0.5)
    precise = refined_peak(config)
    push!(rows, (beta=Float64(beta), gamma=0.1,
        grid_peak=maximum(coarse.I), refined_peak=precise.infected,
        peak_time=precise.time, analytical_peak=precise.analytical,
        final_recovered=coarse.R[end]))
end
scan = DataFrame(rows)
save_csv(group, "beta_scan.csv", scan)

peak_plot = plot(scan.beta, scan.grid_peak; marker=:circle, label="сетка  $\Delta t=0.5$ ",
    xlabel=" $\beta$ ", ylabel="Максимум I", title="Пик эпидемии и точность сетки")
plot!(peak_plot, scan.beta, scan.refined_peak; marker=:diamond, label="уточнённый пик")
final_plot = plot(scan.beta, scan.final_recovered; marker=:circle, color="#27864b",
    label=false, xlabel=" $\beta$ ", ylabel="R(100)", title="Итоговое число переболевших")
save_plot(group, "beta-scan.png", plot(peak_plot, final_plot; layout=(1, 2), size=(1200, 520)))

println("Обработано значений  $\beta$ : $(nrow(scan))")
println("Порог начала роста:  $\beta > \gamma/S_0 = (0.1 / 990)$ ")
println("На обязательной сетке минимальный  $\beta=0.1$  уже значительно выше порога.")
```

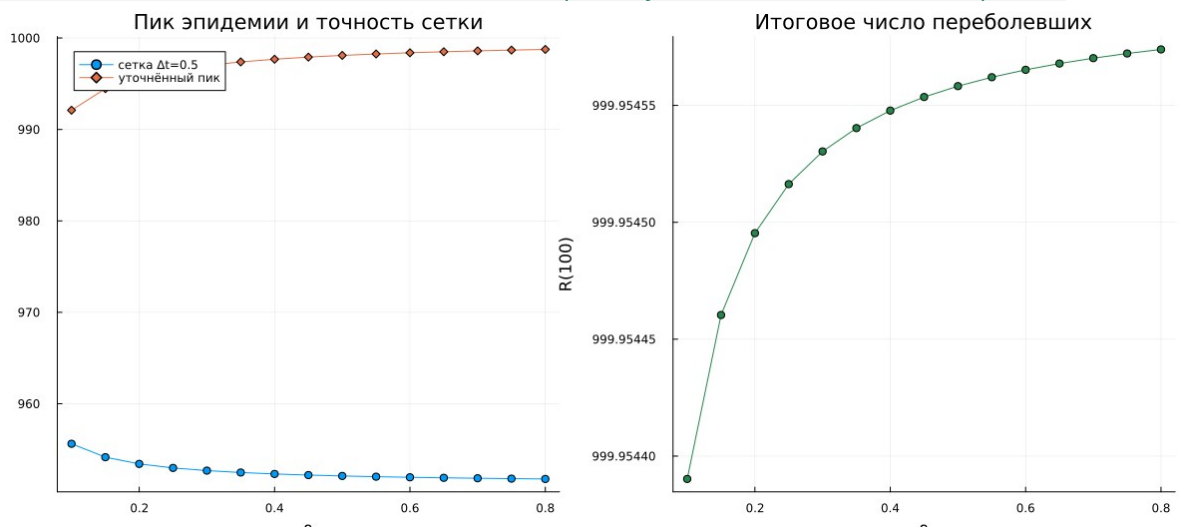


Рисунок 7: Сканирование обязательной сетки β

В выбранном законе βSI начальный рост возможен при $\beta > \gamma / S_0 \approx 0.000101$. Поэтому даже $\beta = 0.1$ лежит выше порога; утверждение об отсутствии эпидемии при этом значении было бы неверным.

7.3 Анимация динамики маркировки

В отличие от ошибочно повторённого сканирования в примере методички этот сценарий строит настоящую анимацию из 501 состояния с шагом 0.2.

```
# # Анимация динамики маркировки
#
# В отличие от ошибочно повторённого сканирования в примере методички этот
# сценарий строит настоящую анимацию из 501 состояния с шагом 0.2.

ENV["GKSwstype"] = "100"
using DataFrames, Plots
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-animation"
trajectory = deterministic_trajectory(SIRConfig(); saveat=0.2)
save_csv(group, "animation_states.csv", trajectory)

animation = @animate for frame in 1:nrow(trajectory)
    animation_plot(trajectory, frame)
end
ensure_outputs(group)
gif(animation, plot_dir(group, "sir-marking.gif"); fps=25)

selected = unique(round.(Int, range(1, nrow(trajectory), length=6)))
frames = [animation_plot(trajectory, i; title="Ключевое состояние $(i)" for i in selected]
save_plot(group, "animation-keyframes.png", plot(frames...; layout=(3, 2), size=(1200, 1350)))
save_csv(group, "animation_manifest.csv", DataFrame(
    frames=[nrow(trajectory)], step=[0.2], duration_seconds=[nrow(trajectory) / 25]))
println("Сформировано кадров: $(nrow(trajectory)); шаг времени: 0.2")
```

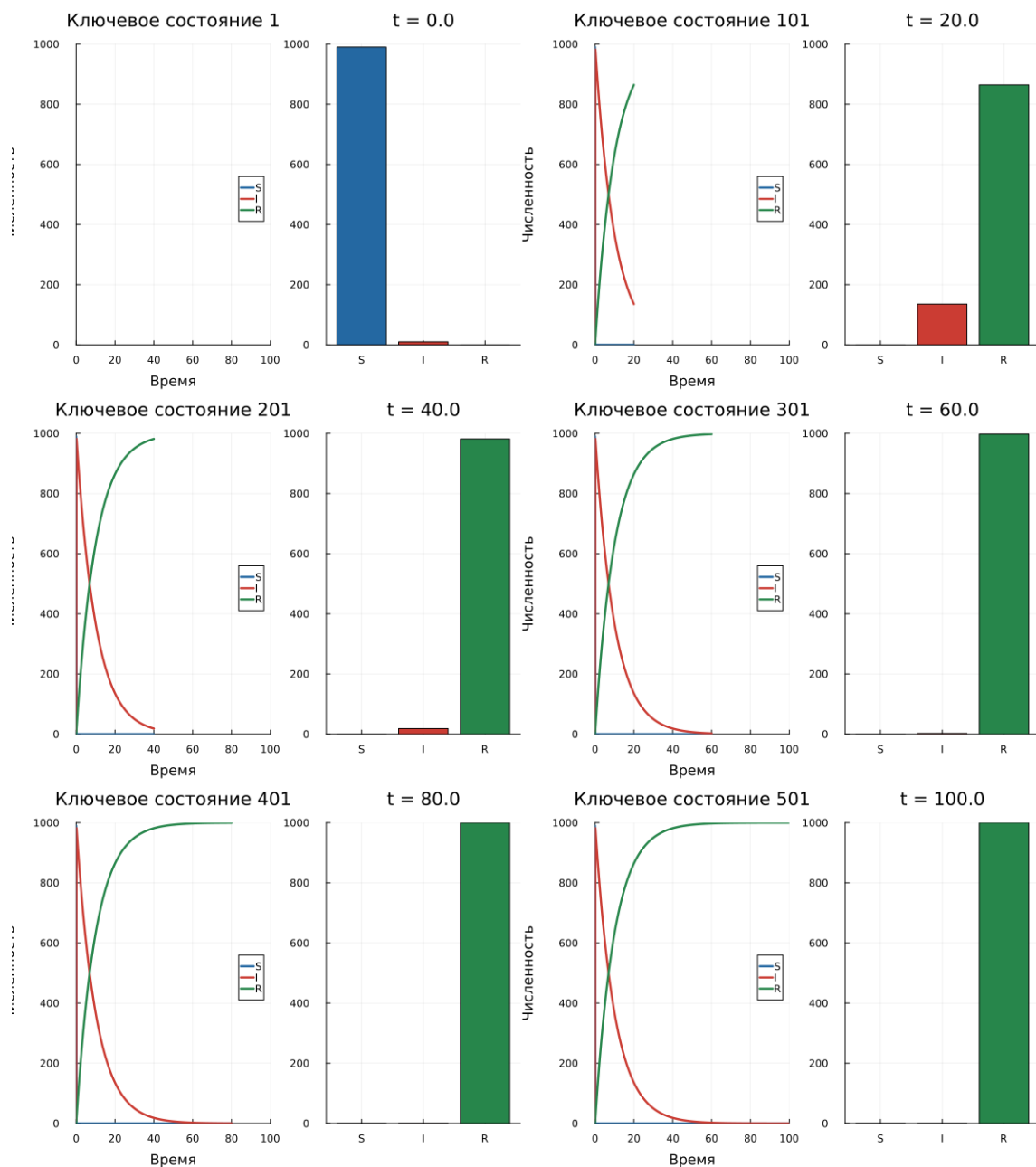


Рисунок 8: Шесть кадров настоящей анимации

7.4 Сводный отчёт по ранее сохранённым данным

Этот сценарий принципиально не запускает модель повторно: все таблицы читаются из CSV, созданных тремя предыдущими обязательными сценариями.

```
# # Сводный отчёт по ранее сохранённым данным
#
```

```
# Этот сценарий принципиально не запускает модель повторно: все таблицы читаются
# из CSV, созданных тремя предыдущими обязательными сценариями.
```

```
ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots, Statistics
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .StudyIO
```

```

group = "sirpetri-report"
det = CSV.read(data_dir("sirpetri-run", "sir_deterministic.csv"), DataFrame)
ssa = CSV.read(data_dir("sirpetri-run", "sir_stochastic_events.csv"), DataFrame)
events = CSV.read(data_dir("sirpetri-run", "event_time_comparison.csv"), DataFrame)
scan = CSV.read(data_dir("sirpetri-scan", "beta_scan.csv"), DataFrame)
manifest = CSV.read(data_dir("sirpetri-animation", "animation_manifest.csv"), DataFrame)

comparison = plot(det.time, det.I; label="ODE", color="#174a7e", linewidth=3,
  xlabel="Время", ylabel="I", title="ODE и SSA")
plot!(comparison, ssa.time, ssa.I; label="SSA, seed=123", color="#c43b3b",
  seriestype=:steppost, alpha=0.75)
difference = plot(events.time, events.difference; label=false, color="#7b3294",
  xlabel="Время события SSA", ylabel="SSA I – ODE I",
  title="Отклонение на собственной сетке событий")
save_plot(group, "method-comparison.png",
  plot(comparison, difference; layout=(2, 1), size=(1100, 850)))

sensitivity = plot(scan.beta, scan.refined_peak; label="уточнённый пик",
  marker=:circle, xlabel="β", ylabel="I_max", title="Чувствительность к β")
plot!(sensitivity, scan.beta, scan.grid_peak; label="максимум на Δt=0.5", marker=:square)
save_plot(group, "sensitivity-summary.png", sensitivity)

summary = DataFrame(
  metric=["RMSE I на событиях SSA", "Среднее |ΔI|", "Кадров анимации", "Точек β"],
  value=[sqrt(mean(events.difference.^2)), mean(abs.(events.difference)),
    manifest.frames[1], nrow(scan)])
save_csv(group, "report_metrics.csv", summary)
println(summary)
println("Отчёт собран только из существующих CSV; повторной симуляции нет.")

```

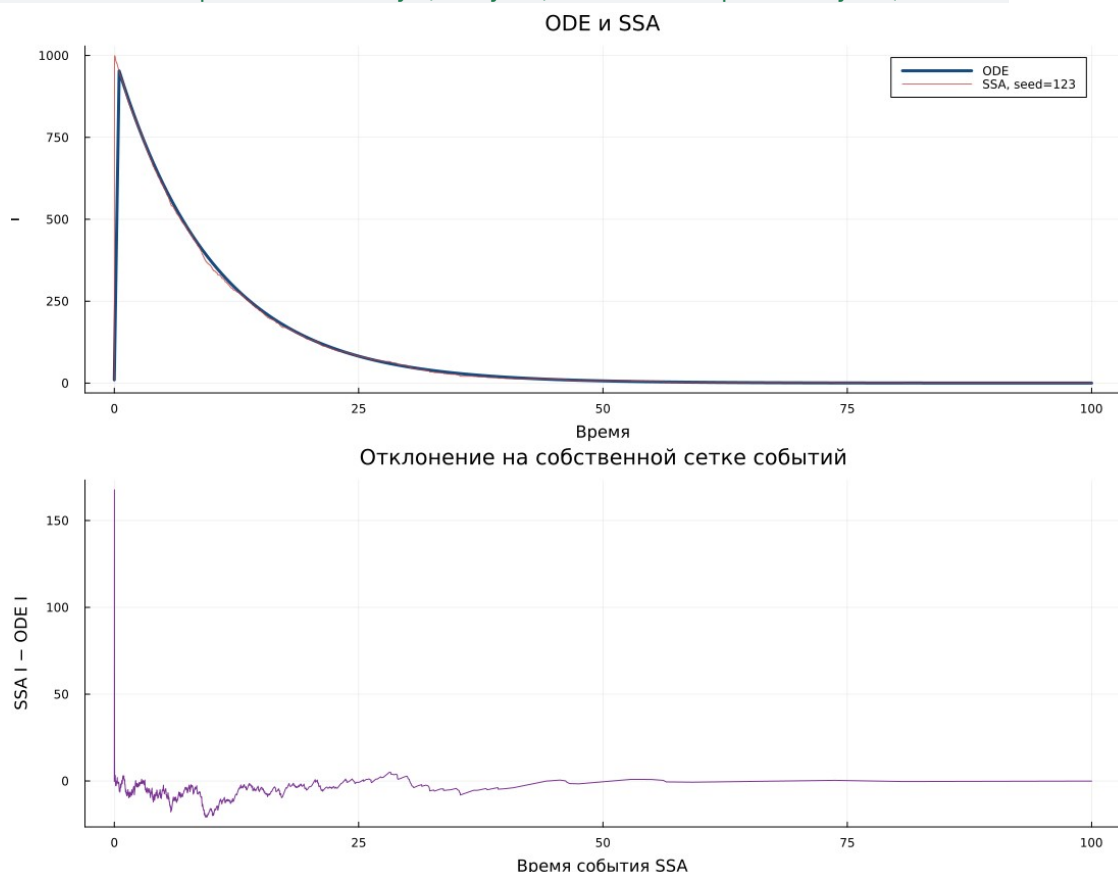


Рисунок 9: Сопоставление ODE и SSA на временах событий

8. Параметрические эксперименты

8.1 Параметрический эксперимент: начальное число инфицированных

Дополнительный эксперимент проверяет, как $I(0)$ влияет на время раннего пика. Для каждого варианта выполнено 12 реальных стохастических повторов.

```
# # Параметрический эксперимент: начальное число инфицированных
#
# Дополнительный эксперимент проверяет, как I(0) влияет на время раннего пика.
# Для каждого варианта выполнено 12 реальных стохастических повторов.

ENV["GKSwstype"] = "100"
using DataFrames, Plots, Statistics
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-initial-study"
rows = NamedTuple[]
det_rows = NamedTuple[]
for initial_i in (5, 10, 25)
    config = SIRConfig(susceptible=1000-initial_i, infected=initial_i)
    exact = refined_peak(config)
    push!(det_rows, (initial_I=initial_i, refined_peak=exact.infected, peak_time=exact.time))
    for replicate in 1:12
        path = stochastic_trajectory(config; seed=6000 + 100initial_i + replicate)
        metrics = trajectory_metrics(path)
        push!(rows, (initial_I=initial_i, replicate=replicate,
                    peak_I=metrics.peak_I, peak_time=metrics.peak_time,
                    final_R=metrics.final_R))
    end
end
replicates = DataFrame(rows)
deterministic = DataFrame(det_rows)
summary = combine(groupby(replicates, :initial_I),
    :peak_I => mean => :mean_peak,
    :peak_I => std => :sd_peak,
    :peak_time => mean => :mean_peak_time,
    :final_R => mean => :mean_final_R)
save_csv(group, "ssa_replicates.csv", replicates)
save_csv(group, "ssa_summary.csv", summary)
save_csv(group, "deterministic_reference.csv", deterministic)

p1 = scatter(replicates.initial_I, replicates.peak_I; group=replicates.initial_I,
    label=false, xlabel="I(0)", ylabel="Пик I", title="12 повторов SSA")
p2 = plot(deterministic.initial_I, deterministic.peak_time; marker=:diamond,
    label=false, xlabel="I(0)", ylabel="Время пика", title="Уточнённое ODE-решение")
save_plot(group, "initial-infection-study.png", plot(p1, p2; layout=(1, 2), size=(1100, 480)))
println("Реальных SSA-прогонов: $(nrow(replicates))")
```

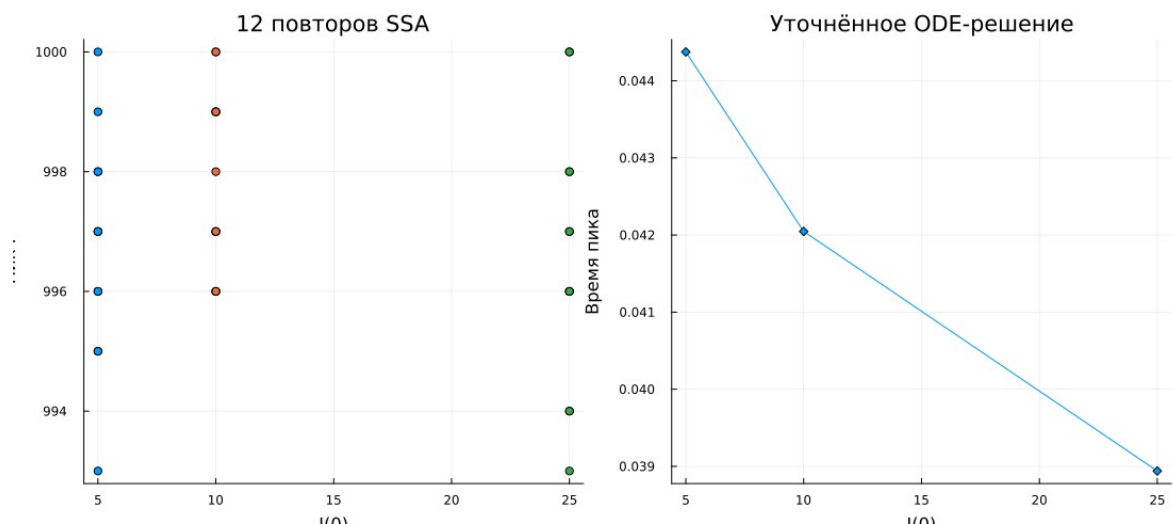


Рисунок 10: Влияние начального числа инфицированных

8.2 Двумерное параметрическое сканирование

Небольшая, но полностью рассчитанная сетка $\beta \times \gamma$ служит разведочным экспериментом. Она не заменяет обязательное сканирование 15 значений β .

```
# # Двумерное параметрическое сканирование
#
# Небольшая, но полностью рассчитанная сетка  $\beta \times \gamma$  служит разведочным
# экспериментом. Она не заменяет обязательное сканирование 15 значений  $\beta$ .

ENV["GKSwstype"] = "100"
using DataFrames, Plots
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-beta-gamma"
rows = NamedTuple{ }
for gamma in (0.05, 0.10, 0.20), beta in (0.10, 0.30, 0.50, 0.80)
    config = SIRConfig(beta=beta, gamma=gamma)
    peak = refined_peak(config)
    trajectory = deterministic_trajectory(config; saveat=0.5)
    push!(rows, (beta=beta, gamma=gamma, peak_I=peak.infected,
        peak_time=peak.time, final_R=trajectory.R[end],
        initial_growth=beta*config.susceptible-gamma))
end
grid = DataFrame(rows)
save_csv(group, "beta-gamma-grid.csv", grid)

p = plot(; xlabel="β", ylabel="Уточнённый пик I", title="Разведочная сетка β×γ")
for subset in groupby(grid, :gamma)
    plot!(p, subset.beta, subset.peak_I; marker=:circle, label="γ=$(subset.gamma[1])")
end
save_plot(group, "beta-gamma-grid.png", p)
println("Рассчитано комбинаций: $(nrow(grid)); сетка предварительная, без интерполяции.")
```

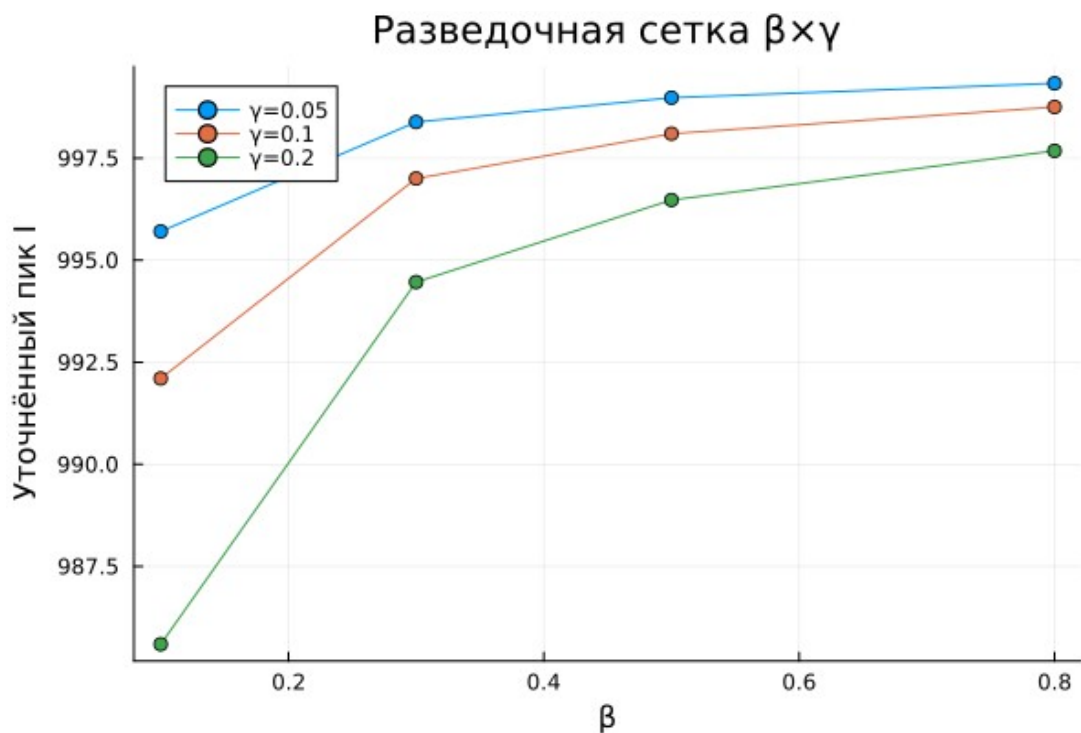


Рисунок 11: Разведочная сетка $\beta \times \gamma$

Сетка 4×3 рассчитана полностью, но является предварительной: по ней не выполняется интерполяция и не оцениваются доверительные интервалы. Это осознанное ограничение дополнительного исследования, которое не затрагивает обязательные 15 точек сканирования.

8.3 Сравнение ключевых состояний для трёх режимов

Чтобы не дублировать базовую GIF-анимацию, параметрический сценарий строит синхронные траектории и панель шести содержательных моментов.

```
# # Сравнение ключевых состояний для трёх режимов
#
# Чтобы не дублировать базовую GIF-анимацию, параметрический сценарий строит
# синхронные траектории и панель шести содержательных моментов.
```

```
ENV["GKSwstype"] = "100"
using DataFrames, Plots
include(joinpath(dirname(Base.active_project()), "src", "SIRPetri.jl"))
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .SIRPetri, .StudyIO

group = "sirpetri-regimes"
cases = [(name="медленнее", beta=0.1, gamma=0.1),
          (name="базовый", beta=0.3, gamma=0.1),
          (name="быстрое выздоровление", beta=0.3, gamma=0.2)]
rows = NamedTuple{[]}()
p = plot(; xlabel="Время", ylabel="I", title="Три режима модели SIR")
for case in cases
    config = SIRConfig(beta=case.beta, gamma=case.gamma)
    trajectory = deterministic_trajectory(config; saveat=0.2)
    plot!(p, trajectory.time, trajectory.I; linewidth=2.5,
          label="$(case.name):  $\beta$ =$(case.beta),  $\gamma$ =$(case.gamma)")
end
```



```

peak = refined_peak(config)
push!(rows, (case=case.name, beta=case.beta, gamma=case.gamma,
            frames=nrow(trajecory), peak_I=peak.infected, peak_time=peak.time))
end
save_csv(group, "regime_manifest.csv", DataFrame(rows))
save_plot(group, "regime-comparison.png", p)
println("Сопоставлено режимов: $(length(cases)); по 501 состоянию в каждом.")

```

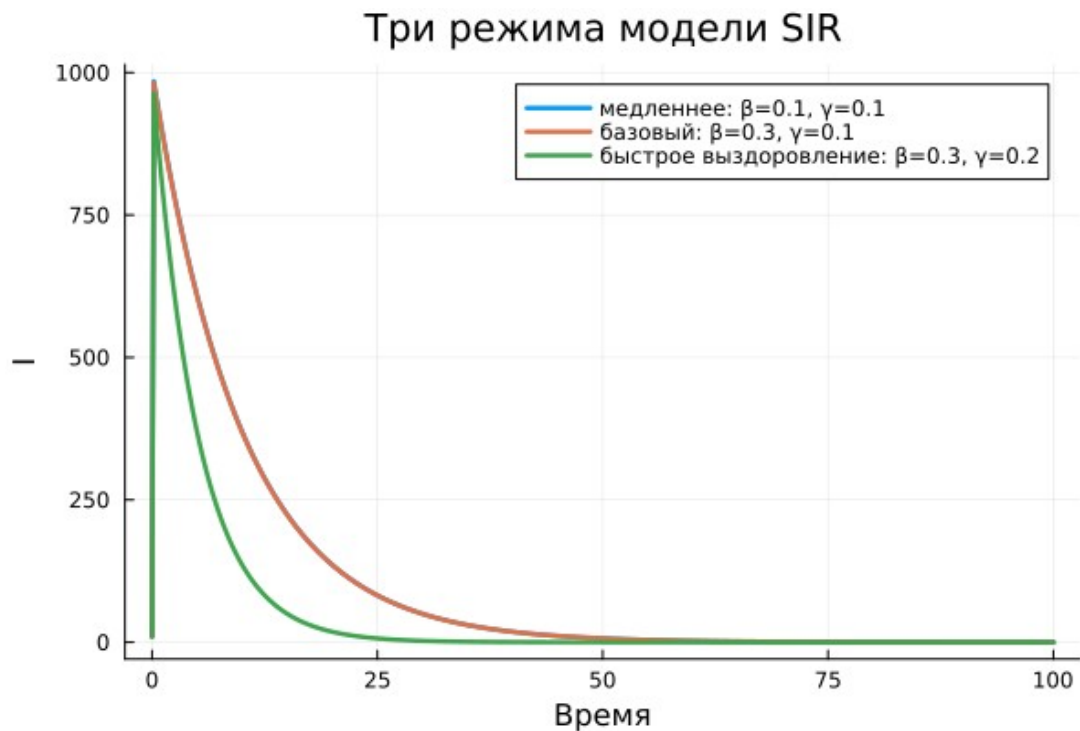


Рисунок 12: Сравнение трёх режимов

8.4 Сводка дополнительных экспериментов

Сценарий читает подготовленные CSV и объединяет результаты, не выполняя моделирование. Ограниченная сетка честно отмечена как разведочная.

```

# # Сводка дополнительных экспериментов
#
# Сценарий читает подготовленные CSV и объединяет результаты, не выполняя
# моделирование. Ограниченная сетка честно отмечена как разведочная.

ENV["GKSwstype"] = "100"
using CSV, DataFrames, Plots
include(joinpath(dirname(Base.active_project()), "src", "StudyIO.jl"))
using .StudyIO

group = "sirpetri-param-report"
summary = CSV.read(data_dir("sirpetri-initial-study", "ssa_summary.csv"), DataFrame)
reference = CSV.read(data_dir("sirpetri-initial-study", "deterministic_reference.csv"), DataFrame)
grid = CSV.read(data_dir("sirpetri-beta-gamma", "beta-gamma-grid.csv"), DataFrame)
regimes = CSV.read(data_dir("sirpetri-regimes", "regime_manifest.csv"), DataFrame)

joined = innerjoin(summary, reference; on=:initial_I)
joined.bias_to_ode = joined.mean_peak .- joined.refined_peak
save_csv(group, "initial-condition-comparison.csv", joined)
save_csv(group, "scope.csv", DataFrame(
    item=["SSA repeats", " $\beta \times \gamma$  combinations", "regimes"],
    value=[sum(fill(12, nrow(summary))), nrow(grid), nrow(regimes)],

```

```
status=["complete", "exploratory", "complete"]]))

p1 = bar(string.(joined.initial_I), joined.bias_to_ode; label=false,
  xlabel="I(0)", ylabel="Средний SSA-пик – ODE-пик", title="Стохастическое отклонение")
p2 = scatter(grid.initial_growth, grid.peak_I; group=grid.gamma,
  xlabel="βS₀–γ", ylabel="Пик I", title="Начальный темп и максимум", legend=:bottomright)
save_plot(group, "parameter-report.png", plot(p1, p2; layout=(1, 2), size=(1100, 480)))
println("Параметрическая сводка собрана из CSV; разведочная сетка: 12 точек.")
```

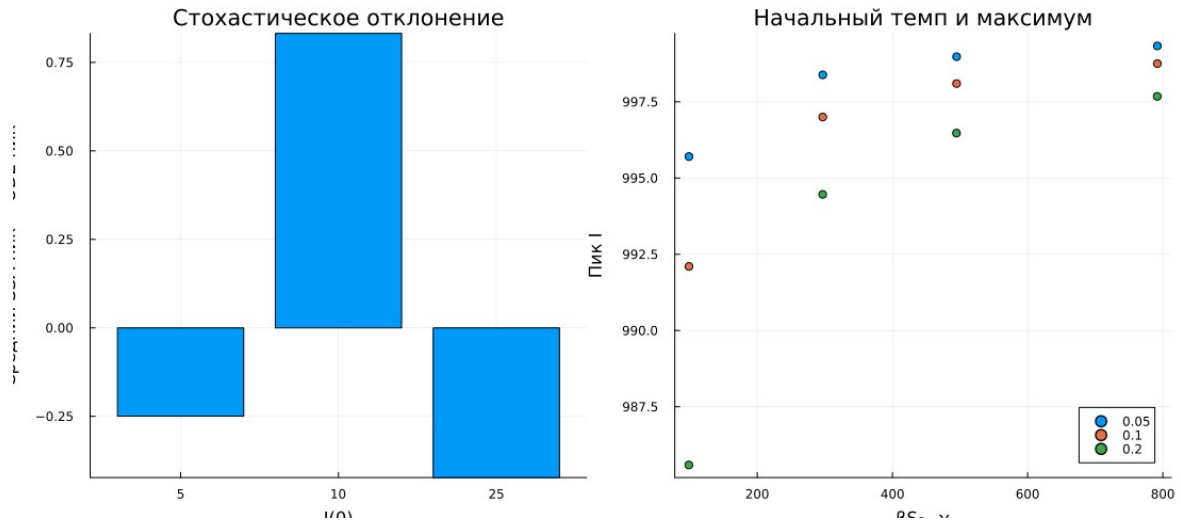


Рисунок 13: Сводка параметрических результатов

9. Воспроизводимость

9.1 Файл окружения

```
name = "PetriSIRStudy"
uuid = "74a87027-89e2-4ee1-b23c-f64be2a853f6"
authors = ["rhktrlwmd"]
version = "6.0.0"

[deps]
AlgebraicPetri = "4f99eebe-17bf-4e98-b6a1-2c4f205a959b"
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
Catlab = "134e5e36-593f-5add-ad60-77f754baafbe"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
OrdinaryDiffEq = "1dea7af3-3e70-54e6-95c3-0bf5283fa5ed"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"
```

```
[compat]
AlgebraicPetri = "0.8"
Catlab = "0.14"
CSV = "0.10"
DataFrames = "1"
IJulia = "1"
Literate = "2"
OrdinaryDiffEq = "7"
Plots = "1"
julia = "1.11"
```

9.2 Автоматизация

```
JULIA ?= julia
SOURCES = scripts/sirpetri_run.jl scripts/sirpetri_scan_parameters.jl scripts/sirpetri_animate.jl
```

```
scripts/sirpetri_report.jl scripts/sirpetri_run_param.jl scripts/sirpetri_scan_parameters_param.jl
scripts/sirpetri_animate_param.jl scripts/sirpetri_report_param.jl
```

.PHONY: setup run tangle notebooks docs test verify all clean-results

setup:

```
$(JULIA) --project=. scripts/setup_environment.jl
```

run:

```
@for source in $(SOURCES); do $(JULIA) --project=. $$source || exit $$?; done
```

tangle:

```
$(JULIA) --project=. scripts/tangle.jl $(SOURCES)
```

notebooks:

```
$(JULIA) --project=. scripts/tangle.jl --execute $(SOURCES)
```

docs:

```
@for source in $(SOURCES); do name=$(basename $$source .jl); \
  (cd notebooks/$$name && quarto render $$name.ipynb --to html --embed-resources --no-
  execute --output $$name.html) || exit $$?; \
  mv notebooks/$$name/$$name.html markdown/$$name/$$name.html; done
```

test:

```
$(JULIA) --project=. test/runtests.jl
```

verify:

```
@test $(find scripts -mindepth 2 -maxdepth 2 -name '*.jl' | wc -l | tr -d ' ') = 8
@test $(find notebooks -mindepth 2 -maxdepth 2 -name '*.ipynb' | wc -l | tr -d ' ') = 8
@test $(find markdown -mindepth 2 -maxdepth 2 -name '*.qmd' | wc -l | tr -d ' ') = 16
@test $(find plots -type f \! -name '*.png' -o -name '*.gif' \) | wc -l | tr -d ' ') -ge 12
@echo "lab06 artifact verification passed"
```

all: run notebooks docs test verify

clean-results:

```
rm -rf data plots notebooks markdown
```

9.3 Подготовка окружения

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
Pkg.instantiate()
using IJulia
IJulia.installkernel("Julia lab06", "--project=$(normpath(joinpath(@__DIR__, "..")))")
Pkg.status()
```

9.4 Генератор литературных форматов

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
using Literate
```

execute = "--execute" in ARGS

sources = filter(arg -> !startswith(arg, "--"), ARGS)

isempty(sources) && (sources = sort(filter(name -> startswith(name, "sirpetri_") && endswith(name, ".jl"), readdir(@__DIR__)))

function report_fragment(source, output)

code = read(source, String)

title = replace(first(filter(line -> startswith(line, "# # "), split(code, '\n'))), "# # " => "")

prose = filter(line -> startswith(line, "#") && !startswith(line, "# #"), split(code, '\n'))

open(output, "w") **do** io

println(io, "## ", title)

println(io, "\n", join(replace.(prose, r"^# ?" => ""), "\n"))

println(io, "\n```\n", code, "\n```\n")

end

end

```

for item in sources
  source = isabspath(item) ? item : joinpath(@_DIR_, basename(item))
  name = splitext(basename(source))[1]
  clean_dir = joinpath(@_DIR_, name)
  notebook_dir = joinpath(@_DIR_, "..", "notebooks", name)
  markdown_dir = joinpath(@_DIR_, "..", "markdown", name)
  foreach(mkpath, [clean_dir, notebook_dir, markdown_dir])
    Literate.script(source, clean_dir; documenter=false)
    Literate.notebook(source, notebook_dir; execute=execute, documenter=false)
    Literate.markdown(source, markdown_dir; documenter=false)
    qmd = joinpath(markdown_dir, "$(name).md")
    isfile(qmd) && mv(qmd, joinpath(markdown_dir, "$(name).qmd"); force=true)
    report_fragment(source, joinpath(markdown_dir, "$(name)-report.qmd"))
  end
println("Generated $(length(sources)) literary scenarios; execute=$(execute)")

```

9.5 Автоматические проверки

```

using Test, DataFrames
include(joinpath(@_DIR_, "..", "src", "SIRPetri.jl"))
using .SIRPetri

@testset "SIR Petri model" begin
  network = build_sir_network()
  @test incidence_matrix(network) == [-1 0; 1 -1; 0 1]
  config = SIRConfig()
  ode = deterministic_trajectory(config; saveat=0.5)
  ssa = stochastic_trajectory(config; seed=123)
  @test nrow(ode) == 201
  @test ode.S + ode.I + ode.R ≈ fill(1000.0, nrow(ode)) atol=1e-7
  @test all(ssa.S + ssa.I + ssa.R .== 1000)
  @test nrow(ssa) > 1000
  peak = refined_peak(config)
  @test abs(peak.infected - peak.analytical) < 1e-5
  @test peak.time < 0.5
  sampled = sample_right_continuous(ssa, [0.0, 0.5, 100.0])
  @test sampled.time == [0.0, 0.5, 100.0]
end

```

10. Обсуждение результатов

При параметрах методички заражение протекает очень быстро из-за интенсивности βSI без деления на N . Значительная часть раннего переходного процесса укладывается между $t=0$ и $t=0.5$, поэтому для сохранённых обзорных таблиц достаточен шаг 0.5, а для пика применено плотное решение и аналитическая проверка.

Стохастическая траектория сравнивается с ODE в собственные моменты событий SSA. Это исключает ошибочную линейную интерполяцию ступенчатой маркировки. Один seed показывает воспроизводимую реализацию, а 36 дополнительных прогонов характеризуют вариацию при трёх начальных условиях.

11. Выводы

Реализована настоящая размеченная сеть Петри SIR и два способа моделирования. Выполнены обязательные базовый расчёт, сканирование 15

значений β , анимация из 501 состояния и отчёт без повторной симуляции. Проверены сохранение популяции, согласованность уточнённого пика с аналитическим инвариантом и правостороннее сопоставление SSA. Дополнительная сетка $\beta \times \gamma$ оставлена компактной и явно обозначена как разведочная.

Список литературы

1. Kermack W. O., McKendrick A. G. [A Contribution to the Mathematical Theory of Epidemics](#) // Proceedings of the Royal Society A. 1927. T. 115, № 772. С. 700–721.
2. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва: Российский университет дружбы народов, 2026.
3. AlgebraicJulia contributors. [AlgebraicPetri.jl: Petri Nets in the Applied Category Theory Ecosystem](#). 2026.
4. Rackauckas C., Nie Q. [DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia](#) // Journal of Open Research Software. 2017. T. 5. С. 15.
5. Gillespie D. T. [Exact Stochastic Simulation of Coupled Chemical Reactions](#) // The Journal of Physical Chemistry. 1977. T. 81, № 25. С. 2340–2361.